

導入數值運算和數據處理庫

```
import numpy as np # 用於數值運算和陣列操作
```

```
import pandas as pd # 用於數據分析和處理表格數據
```

```
import matplotlib.pyplot as plt # 用於繪製圖表和可視化
```

```
import seaborn as sns # 基於 matplotlib 的統計圖表庫，提供更美觀的視覺化
```

導入機器學習相關工具

```
from sklearn.model_selection import train_test_split # 用於分割訓練集和測試集
```

分割訓練集和測試集，是在做機器學習或深度學習時，把原本的一整份資料拆成不同用途的資料集，最常見的是下面這兩種：

1 訓練集 (Training Set)

- 用途：拿來「教模型學習」
- 模型會用這些資料調整參數、找規律
- 可以想成是「上課用的考古題」

2 測試集 (Test Set)

- 用途：拿來「檢查模型學得好不好」
- 不參與訓練，只用來評估結果
- 可以想成是「真正的考試題目」

為什麼一定要分？

如果同一份資料又訓練又測試，模型可能只是「背答案」，看起來準確但實際遇

到新資料就不行，這叫做 **過擬合 (Overfitting)**。

分開之後才能知道：

👉 模型遇到「沒看過的資料」表現如何

常見分割比例

- **80% 訓練 / 20% 測試**（最常見）
 - 70% / 30%
 - 90% / 10%（資料很多時）
-

簡單例子

假設你有 **1000 筆資料**：

- 800 筆 → 訓練集
- 200 筆 → 測試集

模型只用 800 筆學習，最後用 200 筆來打分數。

導入系統和文件操作相關庫

```
import os # 用於操作系統相關功能，如文件路徑處理
```

```
import cv2 # OpenCV 庫，用於圖像處理和電腦視覺任務
```

```
from tqdm import tqdm # 用於顯示進度條，方便追蹤迴圈執行進度
```

```
import shutil # 用於高階文件操作，如複製、移動文件
```

導入深度學習框架 TensorFlow

```
import tensorflow as tf
```

```
print(tf.__version__) # 列印 TensorFlow 版本號，用於確認安裝的版本
```

```
# 從 TensorFlow Keras 導入預訓練模型和相關工具

from tensorflow.keras.applications import EfficientNetB0 # 導入
EfficientNetB0 預訓練模型

from tensorflow.keras.applications.efficientnet import preprocess_input # 導
入 EfficientNet 專用的圖像預處理函數

from tensorflow.keras.preprocessing.image import ImageDataGenerator # 用
於圖像數據增強和批次生成

from tensorflow.keras.callbacks import ModelCheckpoint # 用於在訓練過程
中保存最佳模型
```

這段程式碼主要是導入用於**圖像分類**或**電腦視覺深度學習專案**所需的函式庫，特別是使用 **EfficientNet** 模型進行遷移學習。

```
# ✅ (1/3) 加入計算 ROC/AUC 所需的函式庫

from sklearn.metrics import classification_report, confusion_matrix, roc_curve,
auc

# classification_report: 生成分類報告，包含精確率、召回率、F1 分數等指標

# confusion_matrix: 生成混淆矩陣，用於評估分類模型的性能

# roc_curve: 計算 ROC 曲線的假陽性率和真陽性率

# auc: 計算 ROC 曲線下的面積（Area Under Curve），用於評估模型整體表現


from sklearn.preprocessing import label_binarize

# label_binarize: 將標籤轉換為二元化格式，用於多分類問題的 ROC 曲線計算


from itertools import cycle

# cycle: 創建無限循環的迭代器，常用於重複使用有限的顏色或樣式序列

# ---
```

```
# === 配置區域 ===
```

```
# 設定原始數據集的路徑
```

```
data_dir = r"C:\Users\User\OneDrive\Desktop\Special  
Topic\archive\Brain_Stroke_CT_Dataset"
```

```
# cgan_dir 已移除，因為我們不使用資料增強後的數據
```

```
# cgan_dir is removed as we are not using augmented data
```

```
# 設定結果輸出的路徑（用於保存模型、圖表等）
```

```
output_dir = r"C:\Users\User\OneDrive\Desktop\Special Topic\result"
```

這段程式碼主要是：

1. **導入評估指標函式庫**：用於計算模型性能（ROC 曲線、AUC、混淆矩陣等）
2. **設定路徑配置**：指定數據集和結果輸出的目錄位置
3. **說明不使用 CGAN 增強數據**：僅使用原始數據集進行訓練

```
#  (2/3) 定義新指標圖片的儲存資料夾
```

```
plot_save_dir = r"C:\Users\User\OneDrive\Desktop\Special Topic\result"
```

```
# 設定用於保存評估指標圖表（如 ROC 曲線、混淆矩陣等）的資料夾路徑
```

```
# ---
```

```
# 如果輸出目錄已存在，則刪除舊的目錄及其所有內容
```

```
if os.path.exists(output_dir):
```

```
    shutil.rmtree(output_dir) # 遞迴刪除整個目錄樹
```

```
# 創建新的輸出目錄

os.makedirs(output_dir)

print(f"Output directory created at: {output_dir}") # 顯示輸出目錄的路徑


# ---

# 建立新指標的資料夾（如果已存在則不會報錯）

os.makedirs(plot_save_dir, exist_ok=True) # exist_ok=True 表示目錄存在時不會拋出錯誤

print(f"Additional plots directory created at: {plot_save_dir}") # 顯示圖表保存目錄的路徑


# ---


# === 獲取各類別圖片的路徑 ===

# 建構缺血性中風（Ischemia）圖片資料夾的完整路徑

ischemia_path = os.path.join(data_dir, 'Ischemia', 'PNG')


# 建構出血性中風（Hemorrhagic）圖片資料夾的完整路徑

hemorrhagic_path = os.path.join(data_dir, 'Hemorrhagic', 'PNG')


# 建構正常（Normal）腦部掃描圖片資料夾的完整路徑

normal_path = os.path.join(data_dir, 'Normal', 'PNG')


# 獲取各類別資料夾中的所有圖片檔案名稱列表

normal_images = os.listdir(normal_path) # 正常類別的所有圖片

ischemia_images = os.listdir(ischemia_path) # 缺血性中風類別的所有
```

圖片

```
hemorrhagic_images = os.listdir(hemorrhagic_path) # 出血性中風類別的所有圖片
```

這段程式碼主要功能：

1. **設定並創建輸出目錄**：清空舊的結果資料夾，創建新的乾淨目錄
2. **創建圖表保存目錄**：用於存放模型評估的視覺化圖表
3. **構建數據路徑**：定位三種腦部 CT 掃描類別（正常、缺血性中風、出血性中風）
4. **獲取圖片列表**：讀取每個類別資料夾中的所有圖片檔案名稱

```
# === 原始資料分布統計 ===
```

```
print("\n--- Original Dataset Distribution ---") # 列印標題，顯示原始數據集的分布情況
```

```
# 建立字典來儲存各類別的圖片數量
```

```
original_counts = {
```

```
    "Normal": len(normal_images),          # 正常類別的圖片數量
```

```
    "Ischemia": len(ischemia_images),      # 缺血性中風類別的圖片數量
```

```
    "Hemorrhagic": len(hemorrhagic_images) # 出血性中風類別的圖片數量
```

```
}
```

```
# 遍歷字典，逐一系列印每個類別的圖片數量
```

```
for class_name, count in original_counts.items():
```

```
    print(f"{class_name} Images Count: {count}") # 顯示類別名稱和對應的圖片數量
```

```
# 計算並顯示所有類別的圖片總數
```

```
print(f"Total Original Images: {sum(original_counts.values())}") # sum() 函數加  
總所有類別的數量
```

```
print("-----") # 列印分隔線，使輸出更清晰
```

這段程式碼的主要功能：

1. **統計原始數據集的類別分布**：計算每個類別（正常、缺血性、出血性）的圖片數量
2. **顯示數據摘要**：列印各類別的圖片數量和總數
3. **數據平衡性檢查**：幫助了解數據集是否存在類別不平衡問題

這對於後續的模型訓練很重要，因為不平衡的數據集可能需要特殊處理（如加權、採樣等）。

```
# === 載入圖片路徑和標籤 ===
```

```
imgs = [] # 建立空列表，用於儲存所有圖片的完整路徑
```

```
label = [] # 建立空列表，用於儲存對應的標籤（類別名稱）
```

```
# 使用迴圈遍歷三個類別的圖片列表和對應的標籤名稱
```

```
for img_list, label_name in [(normal_images, "Normal"),
```

```
                              (ischemia_images, "Ischemia"),
```

```
                              (hemorrhagic_images, "Hemorrhagic")]:
```

```
    # 動態獲取該類別對應的資料夾路徑
```

```
    # locals() 返回當前作用域的所有變數字典
```

```
    # 例如：label_name="Normal" 時，會取得 normal_path 變數的值
```

```
    path = locals()[f"{label_name.lower()}_path"]
```

```
# 遍歷該類別的所有圖片檔案名稱

for img in img_list:

    # 組合完整的圖片路徑（資料夾路徑 + 檔案名稱）

    img_path = os.path.join(path, img)

    # 將圖片路徑加入 imgs 列表

    imgs.append(img_path)

    # 將對應的標籤（類別名稱）加入 label 列表

    label.append(label_name)

# 建立 Pandas DataFrame，將圖片路徑和標籤組織成表格形式

df = pd.DataFrame({"Image_path": imgs, "Label": label})

# 顯示載入的資料

print("\nOriginal Data Loaded:") # 提示訊息

print(df.head()) # 顯示 DataFrame 的前 5 筆資料，用於快速檢查
```

這段程式碼的主要功能：

1. **收集所有圖片路徑**：遍歷三個類別的資料夾，獲取每張圖片的完整路徑
2. **建立標籤對應**：為每張圖片分配正確的類別標籤
3. **組織成表格格式**：使用 **Pandas DataFrame** 將路徑和標籤結構化，方便後續處理
4. **資料驗證**：顯示前幾筆資料以確認載入正確

這種資料組織方式便於進行訓練集/測試集分割和模型訓練。

```
# === 繪製並保存各類別的樣本圖片 ===
```

```
def plot_samples(df, class_name, save_name):
```

```
    """
```

```
    繪製指定類別的樣本圖片並保存
```

```
    參數：
```

```
        df: 包含圖片路徑和標籤的 DataFrame
```

```
        class_name: 要繪製的類別名稱（如 "Normal", "Ischemia",  
"Hemorrhagic"）
```

```
        save_name: 保存的檔案名稱
```

```
    """
```

```
    # 創建一個 12x6 英吋的畫布
```

```
    plt.figure(figsize=(12, 6))
```

```
    # 從 DataFrame 中篩選出指定類別的圖片，取前 8 張
```

```
    # enumerate() 用於同時獲取索引和元素值
```

```
    # tqdm() 顯示處理進度條
```

```
    for i, img_path in enumerate(tqdm(df[df['Label'] ==  
class_name]['Image_path'].head(8))):
```

```
        # 使用 OpenCV 讀取圖片
```

```
        img = cv2.imread(img_path)
```

```
        # 將圖片從 BGR 色彩空間轉換為 RGB 色彩空間
```

```
        # OpenCV 預設使用 BGR，matplotlib 使用 RGB，所以需要轉換
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# 在 2 行 4 列的子圖網格中，繪製第 i+1 張圖片
```

```
plt.subplot(2, 4, i + 1)
```

```
# 顯示圖片
```

```
plt.imshow(img)
```

```
# 關閉座標軸顯示，使圖片更清晰
```

```
plt.axis('off')
```

```
# 為每張子圖添加標題（顯示類別名稱）
```

```
plt.title(class_name)
```

```
# 自動調整子圖之間間距，避免重疊
```

```
plt.tight_layout()
```

```
# 將整個圖表保存到指定的輸出目錄
```

```
plt.savefig(os.path.join(output_dir, save_name))
```

```
# 關閉圖表，釋放記憶體
```

```
plt.close()
```

```
# 繪製並保存正常類別的 8 張樣本圖片
```

```
plot_samples(df, "Normal", "sample_images_normal.png")
```

```
# 繪製並保存缺血性中風類別的 8 張樣本圖片
```

```
plot_samples(df, "Ischemia", "sample_images_ischemia.png")
```

```
# 繪製並保存出血性中風類別的 8 張樣本圖片
```

```
plot_samples(df, "Hemorrhagic", "sample_images_hemorrhagic.png")
```

這段程式碼的主要功能：

1. **視覺化樣本數據**：從每個類別中選取 8 張代表性圖片進行展示
2. **色彩空間轉換**：確保圖片在 `matplotlib` 中正確顯示顏色
3. **網格排列**：以 2×4 的網格排列樣本，便於比較
4. **保存結果**：將視覺化結果保存為圖片檔案，方便日後查看和報告使用

這對於理解數據集內容和進行初步的數據探索非常有幫助。

```
# === 繪製類別分佈圖 ===
```

```
# 使用 seaborn 的 countplot 繪製各類別的數量長條圖
```

```
sns.countplot(data=df, x='Label', palette='Set2')
```

```
# data=df: 使用 DataFrame 作為數據來源
```

```
# x='Label': 將標籤作為 x 軸
```

```
# palette='Set2': 使用 Set2 色彩調色板，讓圖表更美觀
```

```
# 設定圖表標題
```

```
plt.title("Distribution of Classes")
```

```
# 將圖表保存到輸出目錄
```

```
plt.savefig(os.path.join(output_dir, "class_distribution.png"))
```

```
# 關閉圖表，釋放記憶體
```

```
plt.close()
```

```
# === 分割數據集 ===
```

```
# 第一次分割：將數據分為訓練集（80%）和測試集（20%）
```

```
train_df, test_df = train_test_split(
```

```
    df,                                # 原始 DataFrame
```

```
    test_size=0.2,                      # 測試集佔 20%
```

```
    random_state=42,                    # 設定隨機種子，確保結果可重現
```

```
    stratify=df['Label']                # 分層抽樣，保持各類別在訓練集和測試集中的比例一致
```

```
)
```

```
# 第二次分割：將訓練集再分為訓練集（80%）和驗證集（20%）
```

```
# 相當於原始數據的 64% 訓練、16% 驗證、20% 測試
```

```
train_df, val_df = train_test_split(
```

```
    train_df,                           # 使用第一次分割後的訓練集
```

```
    test_size=0.2,                       # 驗證集佔訓練集的 20%（即原始數據的 16%）
```

```
    random_state=42,                    # 設定隨機種子
```

```
    stratify=train_df['Label']          # 分層抽樣，保持類別比例
```

```
)
```

```
# 顯示分割後各數據集的樣本數量
```

```
print(f"\nOriginal Split: Train samples: {len(train_df)}, Validation samples:  
{len(val_df)}, Test samples: {len(test_df)}")
```

這段程式碼的主要功能：

1. **視覺化類別分布**：繪製長條圖顯示各類別的數量，檢查數據平衡性
2. **數據集分割**：將數據分為訓練集、驗證集和測試集（約 64%、16%、20%）
3. **分層抽樣**：使用 `stratify` 參數確保各子集保持相同的類別比例
4. **可重現性**：設定 `random_state=42` 確保每次執行得到相同的分割結果

這種分割方式是機器學習中的標準做法，確保模型評估的公正性。

```
# === 數據分布統計 ===
```

```
print("\n--- Data Distribution ---") # 列印標題，顯示數據分布統計資訊
```

```
# 統計訓練集中各類別的數量
```

```
# value_counts() 會計算每個標籤出現的次數，並按降序排列
```

```
train_counts = train_df['Label'].value_counts()
```

```
# 統計驗證集中各類別的數量
```

```
val_counts = val_df['Label'].value_counts()
```

```
# 統計測試集中各類別的數量
```

```
test_counts = test_df['Label'].value_counts()
```

```
# 建立匯總 DataFrame，整合三個數據集的類別分布
```

```
summary_df = pd.DataFrame({
```

```

    'Training': train_counts,      # 訓練集的類別數量

    'Validation': val_counts,      # 驗證集的類別數量

    'Testing': test_counts        # 測試集的類別數量

}).fillna(0).astype(int)

# fillna(0): 將缺失值 (NaN) 填充為 0，避免某些類別在特定數據集中不存在
時出現 NaN

# astype(int): 將浮點數轉換為整數，使數據更易讀


# 新增 'Total' 欄位，計算每個類別在所有數據集中的總數量

# axis=1 表示沿著列的方向（橫向）求和

summary_df['Total'] = summary_df.sum(axis=1)


# 新增 'Total' 列，計算每個數據集的總樣本數

# axis=0 表示沿著行的方向（縱向）求和

# loc['Total'] 會在 DataFrame 底部新增一列名為 'Total' 的統計資料

summary_df.loc['Total'] = summary_df.sum()


# 列印完整的數據分布統計表

print(summary_df)

...

```

這段程式碼的主要功能：

1. ****統計各數據集的類別分布****：分別計算訓練集、驗證集、測試集中每個類別的數量
2. ****建立匯總表格****：將三個數據集的統計結果整合成一個易讀的表格

3. **計算總計**：

- 橫向：每個類別在所有數據集中的總數
- 縱向：每個數據集的總樣本數

4. **數據驗證**：確認分層抽樣是否成功保持類別比例一致

輸出範例可能如下：

...

	Training	Validation	Testing	Total
Hemorrhagic	400	100	125	625
Ischemia	640	160	200	1000
Normal	320	80	100	500
Total	1360	340	425	2125

```
# === 建立數據生成器 ===
```

```
# 數據生成器用於批次載入和預處理圖片，避免一次性將所有圖片載入記憶體
```

```
# --- 訓練集數據生成器 ---
```

```
# 建立訓練集的 ImageDataGenerator
```

```
train_datagen = ImageDataGenerator(
```

```
    preprocessing_function=preprocess_input # 使用 EfficientNet 專用的預  
    處理函數（標準化像素值）
```

```
)
```

```
# 從 DataFrame 建立訓練數據的流式生成器
```

```
train_generator = train_datagen.flow_from_dataframe(
```

```

train_df,                # 訓練集 DataFrame

x_col='Image_path',      # 指定圖片路徑的欄位名稱

y_col='Label',           # 指定標籤的欄位名稱

target_size=(224, 224),  # 將所有圖片調整為 224x224 像素
( EfficientNetB0 的輸入尺寸 )

batch_size=32,           # 每批次處理 32 張圖片

class_mode='categorical' # 分類模式為多類別 ( one-hot encoding )

)

# --- 驗證集數據生成器 ---

# 建立驗證集的 ImageDataGenerator

val_datagen = ImageDataGenerator(

    preprocessing_function=preprocess_input # 使用相同的預處理函數

)

# 從 DataFrame 建立驗證數據的流式生成器

val_generator = val_datagen.flow_from_dataframe(

    val_df,                # 驗證集 DataFrame

    x_col='Image_path',    # 圖片路徑欄位

    y_col='Label',         # 標籤欄位

    target_size=(224, 224), # 圖片尺寸調整為 224x224

    batch_size=32,         # 批次大小為 32

    class_mode='categorical' # 多類別分類模式

)

```



```
# --- 測試集數據生成器 ---

# 建立測試集的 ImageDataGenerator

test_datagen = ImageDataGenerator(

    preprocessing_function=preprocess_input # 使用相同的預處理函數

)

# 從 DataFrame 建立測試數據的流式生成器

test_generator = test_datagen.flow_from_dataframe(

    test_df,                                # 測試集 DataFrame

    x_col='Image_path',                     # 圖片路徑欄位

    y_col='Label',                           # 標籤欄位

    target_size=(224, 224),                 # 圖片尺寸調整為 224x224

    batch_size=32,                           # 批次大小為 32

    class_mode='categorical',               # 多類別分類模式

    shuffle=False                            # 不打亂順序，確保預測結果與原始標籤
    對應（用於評估）

)
```

這段程式碼的主要功能：

1. **建立數據生成器**：使用 `ImageDataGenerator` 進行圖片的即時載入和預處理
2. **統一預處理**：所有數據集使用 `EfficientNet` 的標準預處理函數
3. **批次處理**：以批次方式載入圖片，節省記憶體
4. **尺寸標準化**：將所有圖片調整為模型要求的 `224×224` 尺寸
5. **測試集特殊設定**：

- `shuffle=False`：保持數據順序，確保預測結果與真實標籤正確對應
- 這對於後續的模型評估和混淆矩陣計算非常重要

```
# === 建立 EfficientNet 模型 ===
```

```
# 載入預訓練的 EfficientNetB0 作為基礎模型
```

```
base_model = EfficientNetB0(
```

```
    include_top=False,          # 不包含頂層的全連接層（分類層），方便  
    自定義輸出層
```

```
    weights='imagenet',        # 使用在 ImageNet 數據集上預訓練的權  
    重
```

```
    input_shape=(224, 224, 3)   # 輸入圖片的尺寸（224x224 像素，3 個顏  
    色通道 RGB）
```

```
)
```

```
# 凍結基礎模型的所有層，防止在初始訓練時更新這些權重
```

```
base_model.trainable = False
```

```
# --- 自定義模型頂層 ---
```

```
x = base_model.output          # 取得基礎  
模型的輸出
```

```
x = tf.keras.layers.GlobalAveragePooling2D()(x)    # 全局平均池化層，  
將特徵圖壓縮成一維向量
```

```
x = tf.keras.layers.Dense(128, activation='relu')(x)    # 全連接層，128 個神  
經元，ReLU 激活函數
```

```
outputs = tf.keras.layers.Dense(3, activation='softmax')(x)    # 輸出層，3 個類
```

別 (Normal, Ischemia, Hemorrhagic)，softmax 用於多類別分類

建立完整模型（從基礎模型的輸入到自定義的輸出）

```
model = tf.keras.Model(inputs=base_model.input, outputs=outputs)
```

--- 解凍部分層進行微調 ---

```
base_model.trainable = True # 允許基礎模型的層可以訓練
```

```
for layer in base_model.layers[:150]: # 凍結前 150 層
```

```
    layer.trainable = False # 只微調後面的層，保留底層的通用特徵
```

--- 編譯模型 ---

```
model.compile(
```

```
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # 使用 Adam  
    優化器，學習率設為 0.00001（較小，適合微調）
```

```
    loss='categorical_crossentropy', # 多類別分類的損失函數
```

```
    metrics=['accuracy'] # 評估指標為準確率
```

```
)
```

顯示模型結構摘要（層數、參數量等）

```
model.summary()
```

--- 設定模型檢查點 ---

```
# 定義最佳模型的保存路徑

checkpoint_filepath = os.path.join(output_dir, "best_finetuned_model.keras")


# 建立 ModelCheckpoint 回調函數

checkpoint_callback = ModelCheckpoint(

    filepath=checkpoint_filepath,    # 保存路徑

    save_best_only=True,            # 只保存驗證準確率最高的模型

    monitor="val_accuracy",        # 監控驗證集的準確率

    mode="max",                    # 當 val_accuracy 最大時保存

    verbose=1                      # 顯示保存訊息

)


# --- 開始訓練（微調） ---

print("\n--- Starting Fine-tuning ---")

history_finetune = model.fit(

    train_generator,                # 訓練數據生成器

    validation_data=val_generator,  # 驗證數據生成器

    epochs=100,                    # 訓練 100 個週期

    callbacks=[checkpoint_callback] # 使用檢查點回調，自動保存最佳模型

)


print("\n--- Fine-tuning Finished ---")

print(f"✅ Best model saved to '{checkpoint_filepath}'") # 顯示最佳模型的保存位置
```

這段程式碼的主要功能：

1. **遷移學習**：使用預訓練的 **EfficientNetB0** 作為特徵提取器
2. **自定義分類頭**：添加全局池化層、全連接層和輸出層以適應 **3** 類別分類任務
3. **微調策略**：
 - 先凍結所有基礎層
 - 然後解凍後面的層（從第 **150** 層開始）進行微調
 - 使用較小的學習率（**1e-5**）避免破壞預訓練權重
4. **自動保存最佳模型**：根據驗證準確率自動保存表現最好的模型
5. **訓練 100 個週期**：進行充分的訓練以獲得最佳性能

=== 繪製並保存訓練過程的準確率和損失曲線 ===

從訓練歷史記錄中提取訓練集的準確率

acc = history_finetune.history['accuracy']

從訓練歷史記錄中提取驗證集的準確率

val_acc = history_finetune.history['val_accuracy']

從訓練歷史記錄中提取訓練集的損失值

loss = history_finetune.history['loss']

從訓練歷史記錄中提取驗證集的損失值

val_loss = history_finetune.history['val_loss']

```
# --- 繪製雙子圖 ---

# 創建一個 12x5 英吋的畫布

plt.figure(figsize=(12, 5))


# 第一個子圖：準確率曲線

plt.subplot(1, 2, 1) # 1 行 2 列的第 1 個位置

plt.plot(acc, label='Training Accuracy') # 繪製訓練準確率曲線

plt.plot(val_acc, label='Validation Accuracy') # 繪製驗證準確率曲線

plt.legend(loc='lower right') # 在右下角顯示圖例

plt.title('Training and Validation Accuracy') # 設定子圖標題


# 第二個子圖：損失曲線

plt.subplot(1, 2, 2) # 1 行 2 列的第 2 個位置

plt.plot(loss, label='Training Loss') # 繪製訓練損失曲線

plt.plot(val_loss, label='Validation Loss') # 繪製驗證損失曲線

plt.legend(loc='upper right') # 在右上角顯示圖例

plt.title('Training and Validation Loss') # 設定子圖標題


# 保存整個圖表到輸出目錄

plt.savefig(os.path.join(output_dir, "training_history.png"))


# 關閉圖表，釋放記憶體

plt.close()
```

這段程式碼的主要功能：

1. **提取訓練歷史數據**：從 `history_finetune` 物件中取得每個 `epoch` 的準確率和損失值

2. **視覺化訓練過程**：

- 左圖：顯示訓練集和驗證集的準確率變化趨勢
- 右圖：顯示訓練集和驗證集的損失值變化趨勢

3. **診斷模型表現**：

- 檢查是否有過擬合（訓練和驗證曲線差距過大）
- 確認模型是否收斂（曲線趨於平穩）

4. **保存結果**：將訓練歷史圖表保存為圖片，方便日後分析和報告

這些曲線是評估模型訓練品質的重要工具！

```
# === 評估模型 ===
```

```
# 提示訊息：載入訓練過程中保存的最佳模型
```

```
print("\nLoading best model for final evaluation...")
```

```
# 從檔案中載入先前保存的最佳模型（基於驗證準確率）
```

```
best_model = tf.keras.models.load_model(checkpoint_filepath)
```

```
# --- 在測試集上評估模型性能 ---
```

```
# evaluate() 方法會返回損失值和準確率
```

```
val_loss, val_accuracy = best_model.evaluate(test_generator)
```

```
# test_generator: 測試集數據生成器
```

```
# 返回值：
```

```
# - val_loss: 測試集的損失值（越小越好）
```

```
# - val_accuracy: 測試集的準確率（0 到 1 之間的數值）

# 顯示測試集的評估結果

print(f"Evaluation on Test Set: loss is {val_loss:.4f}, accuracy is
{val_accuracy*100:.2f}%")

# {val_loss:.4f}: 顯示損失值，保留 4 位小數

# {val_accuracy*100:.2f}%: 將準確率轉換為百分比，保留 2 位小數
```

這段程式碼的主要功能：

1. **載入最佳模型**：使用訓練過程中自動保存的驗證準確率最高的模型
2. **測試集評估**：在從未見過的測試集上評估模型的真實性能
3. **輸出關鍵指標**：
 - **損失值（Loss）**：衡量預測與真實標籤的差距
 - **準確率（Accuracy）**：正確分類的樣本比例

為什麼要載入最佳模型？

- 訓練 100 個 epoch 後，模型可能在後期出現過擬合
- 載入驗證準確率最高時的模型，確保使用性能最佳的版本進行最終評估
- 測試集的準確率是模型泛化能力的最終指標

```
# === 生成分類報告和混淆矩陣 ===
```

```
# --- 預測測試集 ---
```

```
# 使用最佳模型對測試集進行預測，返回每個類別的機率值
```

```
y_pred_probs = best_model.predict(test_generator)
```

```
# 形狀為 (樣本數, 3)，每一行包含三個類別的預測機率
```



```
# 將機率值轉換為類別索引（選擇機率最高的類別）

y_pred = np.argmax(y_pred_probs, axis=1)

# axis=1 表示在每一行中找最大值的索引

# 例如：[0.1, 0.7, 0.2] -> 1（第二個類別）


# 獲取測試集的真实標籤（類別索引）

y_true = test_generator.classes


# 獲取類別名稱列表（如 ['Hemorrhagic', 'Ischemia', 'Normal']）

class_names = list(test_generator.class_indices.keys())


# 獲取類別總數

n_classes = len(class_names)


# --- 生成分類報告 ---

print("\nClassification Report:")

# classification_report 生成詳細的分類指標報告

report = classification_report(

    y_true,                # 真實標籤

    y_pred,                # 預測標籤

    target_names=class_names # 類別名稱（用於顯示在報告中）

)


# 報告包含：精確率 (Precision)、召回率 (Recall)、F1-score、支持度 (Support)
```

```
print(report)

# --- 生成混淆矩陣 ---

# 計算混淆矩陣（顯示預測與實際的對應關係）

cm = confusion_matrix(y_true, y_pred)

# 返回一個 n_classes x n_classes 的矩陣

# cm[i][j] 表示實際類別 i 被預測為類別 j 的次數


# 繪製混淆矩陣熱力圖

plt.figure(figsize=(8, 6)) # 設定畫布大小

sns.heatmap(

    cm,                                # 混淆矩陣數據

    annot=True,                        # 在每個格子中顯示數值

    fmt='d',                           # 格式為整數 (decimal)

    cmap='Blues',                      # 使用藍色漸層配色

    xticklabels=class_names,          # x 軸標籤（預測類別）

    yticklabels=class_names           # y 軸標籤（實際類別）

)

plt.title('Confusion Matrix')         # 設定標題

plt.ylabel('Actual Label')             # y 軸標題（實際標籤）

plt.xlabel('Predicted Label')          # x 軸標題（預測標籤）


# 保存混淆矩陣圖到輸出目錄

plt.savefig(os.path.join(output_dir, "confusion_matrix.png"))
```

```
plt.close() # 關閉圖表，釋放記憶體
```

```
# 顯示保存路徑
```

```
print(f"Confusion matrix plot saved to '{os.path.join(output_dir, 'confusion_matrix.png')}'")
```

這段程式碼的主要功能：

1. 模型預測：

- 獲取每個樣本的類別機率
- 選擇機率最高的類別作為最終預測

2. 分類報告：

- **Precision (精確率)**：預測為某類別的樣本中，實際為該類別的比例
- **Recall (召回率)**：實際為某類別的樣本中，被正確預測的比例
- **F1-score**：精確率和召回率的調和平均
- **Support**：每個類別的實際樣本數

3. 混淆矩陣：

- 對角線：正確分類的數量
- 非對角線：錯誤分類的情況
- 幫助識別模型容易混淆哪些類別

這些評估指標對於理解模型在不同類別上的表現非常重要！

```
# ✅ (3/3) 新增指標計算 (Specificity, NPV, ROC-AUC)
```

```
# ---
```

```
# 建立列表用於儲存要寫入檔案的額外評估指標
```

```
report_lines_to_add = ["\n\n--- Additional Metrics (One-vs-Rest) ---"]
```

One-vs-Rest: 將多類別問題視為多個二元分類問題（每次將一個類別視為正類，其餘視為負類）

```
# === 1. 計算特異度 (Specificity) 和 陰性預測值 (NPV) ===  
  
print("\n--- Calculating Specificity and NPV (per-class) ---")
```

建立空列表，用於儲存每個類別的特異度和 NPV

```
specificities = [] # 特異度列表
```

```
npvs = []          # 陰性預測值列表
```

遍歷每個類別，計算其對應的指標

```
for i in range(n_classes):
```

```
    # --- 計算混淆矩陣的四個基本值 ---
```

```
    # TP (True Positive, 真陽性): 混淆矩陣對角線上的值
```

```
    # 實際為類別 i 且被正確預測為類別 i 的樣本數
```

```
    #  $TP = cm[i, i]$ 
```

```
    # FP (False Positive, 假陽性): 第 i 列的總和減去 TP
```

```
    # 實際不是類別 i，但被錯誤預測為類別 i 的樣本數
```

```
     $FP = cm[:, i].sum() - cm[i, i]$ 
```

```
    # FN (False Negative, 假陰性): 第 i 行的總和減去 TP
```

```
    # 實際是類別 i，但被錯誤預測為其他類別的樣本數
```

```
FN = cm[i, :].sum() - cm[i, i]
```

```
# TN (True Negative, 真陰性): 總樣本數減去 (TP + FP + FN)
```

```
# 實際不是類別 i 且被正確預測為其他類別的樣本數
```

```
TN = cm.sum() - (cm[i, i] + FP + FN)
```

```
# --- 計算 Specificity (特異度) ---
```

```
# Specificity = TN / (TN + FP)
```

```
# 表示：在所有實際為負類的樣本中，被正確預測為負類的比例
```

```
# 衡量模型識別「非該類別」的能力
```

```
specificity = TN / (TN + FP) if (TN + FP) > 0 else 0 # 避免除以零
```

```
specificities.append(specificity) # 加入列表
```

```
# --- 計算 NPV (Negative Predictive Value, 陰性預測值) ---
```

```
# NPV = TN / (TN + FN)
```

```
# 表示：在所有被預測為負類的樣本中，實際為負類的比例
```

```
# 衡量模型「排除該類別」的可信度
```

```
npv = TN / (TN + FN) if (TN + FN) > 0 else 0 # 避免除以零
```

```
npvs.append(npv) # 加入列表
```

```
# --- 顯示結果 ---
```

```
print(f"Class: {class_names[i]}")
```

```
print(f" Specificity: {specificity:.4f}") # 顯示特異度，保留 4 位小數
```

```
print(f" NPV (Negative Predictive Value): {npv:.4f}") # 顯示 NPV
```

```
# 準備寫入報告的文字內容
```

```
report_lines_to_add.append(f"Class: {class_names[i]}")
```

```
report_lines_to_add.append(f"    Specificity: {specificity:.4f}")
```

```
report_lines_to_add.append(f"    NPV:                {npv:.4f}")
```

這段程式碼的主要功能：

1. 計算額外的評估指標：

- **Specificity (特異度)**：正確識別「非該類別」的能力
- **NPV (陰性預測值)**：預測為「非該類別」時的可信度

2. 指標意義：

- 高 **Specificity**：模型很少將其他類別誤判為該類別（假陽性率低）
- 高 **NPV**：當模型說「不是這個類別」時，通常是對的

3. 醫療應用的重要性：

- 在腦中風診斷中，這些指標非常關鍵
- 例如：高 **NPV** 表示當模型判定「非出血性中風」時，患者真的不是出血性中風的機率很高

4. **One-vs-Rest** 策略：

- 將每個類別依次視為正類，其餘視為負類
- 分別計算每個類別的二元分類指標

```
# === 2. 計算並繪製 ROC-AUC 曲線 ===
```

```
print("\n--- Calculating and Plotting ROC-AUC Curve ---")
```

```
# --- 將真實標籤進行 One-Hot 編碼（二值化） ---
```

```
y_true_bin = label_binarize(y_true, classes=range(n_classes))
```

```
# label_binarize: 將多類別標籤轉換為二元格式

# 例如：類別 [0, 1, 2] 轉換為：

# 類別 0 -> [1, 0, 0]

# 類別 1 -> [0, 1, 0]

# 類別 2 -> [0, 0, 1]

# 這是計算多類別 ROC 曲線的必要步驟（One-vs-Rest 策略）


# --- 初始化字典來儲存每個類別的 ROC 相關數據 ---

fpr = dict()      # False Positive Rate（假陽性率）字典

tpr = dict()      # True Positive Rate（真陽性率）字典

roc_auc = dict()  # AUC（ROC 曲線下面積）字典


# --- 為每個類別計算 ROC 曲線和 AUC ---

for i in range(n_classes):

    # 計算 ROC 曲線

    fpr[i], tpr[i], _ = roc_curve(

        y_true_bin[:, i],    # 第 i 個類別的真實標籤（0 或 1）

        y_pred_probs[:, i]   # 第 i 個類別的預測機率（0 到 1 之間）

    )

    # roc_curve 返回：

    # - fpr: 假陽性率（在不同閾值下）

    # - tpr: 真陽性率（在不同閾值下）

    # - _: 閾值（這裡不需要使用，所以用 _ 忽略）
```

計算 AUC (Area Under Curve, ROC 曲線下面積)

```
roc_auc[i] = auc(fpr[i], tpr[i])
```

AUC 值範圍：0 到 1

- 1.0：完美分類器

- 0.5：隨機猜測

- 越接近 1，模型性能越好

這段程式碼的主要功能：

1. One-Hot 編碼：

- 將多類別標籤轉換為二元格式
- 每個類別獨立處理 (One-vs-Rest)

2. ROC 曲線計算：

- **FPR (假陽性率)**：在不同閾值下，錯誤預測為正類的比例
- **TPR (真陽性率/召回率)**：在不同閾值下，正確預測為正類的比例
- ROC 曲線展示 TPR 和 FPR 在不同閾值下的權衡關係

3. AUC 計算：

- 衡量模型整體的分類性能
- 不受分類閾值影響
- 常用於比較不同模型的優劣

4. 醫療應用意義：

- 在腦中風診斷中，AUC 可以評估模型區分不同類型中風的能力
- 高 AUC 表示模型能有效區分正常、缺血性和出血性中風

--- 計算 micro-average (微觀平均) ROC 曲線和 AUC ---

micro-average: 將所有類別的預測結果合併計算，不區分類別


```

# 適合用於整體性能評估，特別是在類別不平衡時

# 計算 micro-average 的 FPR 和 TPR
fpr["micro"], tpr["micro"], _ = roc_curve(
    y_true_bin.ravel(),      # 將 One-Hot 編碼的真實標籤攤平成一維陣列
    y_pred_probs.ravel()    # 將預測機率攤平成一維陣列
)

# ravel(): 將多維陣列展平為一維
# 例如：[[1,0,0], [0,1,0]] -> [1,0,0,0,1,0]
# 這樣處理後，所有類別的樣本都會一起計算 ROC 曲線

# 計算 micro-average 的 AUC 值
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

# 這個 AUC 代表模型在所有類別上的整體分類性能

# --- 繪製所有 ROC 曲線 ---
plt.figure(figsize=(10, 8)) # 創建 10x8 英吋的畫布

# 繪製 Micro-average ROC 曲線
plt.plot(
    fpr["micro"],          # x 軸：
    假陽性率              # y 軸：
    tpr["micro"],          # y 軸：
    真陽性率
    label=f'Micro-average ROC (area = {roc_auc["micro"]:0.4f})', # 圖例標籤

```

(顯示 AUC 值)

```
        color='deeppink',                                # 線條顏色：深粉紅色
        linestyle=':',                                    # 線條樣式：點線
        linewidth=4                                       # 線條寬度：4
    )
```

這段程式碼的主要功能：

1. Micro-average ROC 的特點：

- **合併所有類別**：將所有類別的預測視為一個整體
- **忽略類別差異**：不考慮各類別的樣本數量差異
- **整體性能指標**：提供模型在所有類別上的平均表現

2. 與 Macro-average 的區別：

- **Micro-average**：先合併所有預測，再計算指標（受樣本數量影響）
- **Macro-average**：先計算每個類別的指標，再取平均（每個類別權重相同）

3. 醫療應用意義：

- **Micro-average AUC** 反映模型整體的診斷準確性
- 如果三種中風類型的樣本數量不同，**micro-average** 會偏向樣本數較多的類別

4. 視覺化設計：

- 使用深粉紅色點線突出顯示整體性能
- 較粗的線條（**linewidth=4**）使其在圖表中更醒目

--- 繪製每個類別的 ROC 曲線 ---

```

# 創建顏色循環器，用於為不同類別分配顏色

colors = cycle(['aqua', 'darkorange', 'cornflowerblue'])

# cycle(): 創建無限循環的迭代器，當顏色用完時會自動重複

# 三種顏色：水藍色、深橙色、矢車菊藍色


# 遍歷每個類別，繪製其對應的 ROC 曲線

for i, color in zip(range(n_classes), colors):

    # zip(): 將類別索引和顏色配對

    # 例如：(0, 'aqua'), (1, 'darkorange'), (2, 'cornflowerblue')


    plt.plot(

        fpr[i],                # x 軸：第 i 個類別的假陽性率

        tpr[i],                # y 軸：第 i 個類別的真陽性率

        color=color,           # 線條顏色

        lw=2,                  # 線條寬度：2

        label=f'ROC curve of class {class_names[i]} (area = {roc_auc[i]:0.4f})'

        # 圖例標籤：顯示類別名稱和對應的 AUC 值（保留 4 位小數）

    )


# --- 繪製對角線（隨機猜測的基準線） ---

plt.plot([0, 1], [0, 1], 'k--', lw=2)

# [0, 1], [0, 1]: 從點 (0,0) 到點 (1,1) 的對角線

# 'k--': k 表示黑色 (black)，-- 表示虛線

# lw=2: 線條寬度為 2

```

```
# 這條線代表 AUC = 0.5 的隨機分類器（無鑑別能力）

# --- 設定座標軸範圍 ---

plt.xlim([0.0, 1.0])    # x 軸範圍：0 到 1

plt.ylim([0.0, 1.05])   # y 軸範圍：0 到 1.05（稍微超出以留白）

# --- 設定座標軸標籤 ---

plt.xlabel('False Positive Rate') # x 軸標籤：假陽性率（FPR）

plt.ylabel('True Positive Rate')  # y 軸標籤：真陽性率（TPR）/ 召回率

# --- 設定圖表標題 ---

plt.title('Multi-class Receiver Operating Characteristic (ROC) Curve')

# 多類別接收者操作特徵曲線

# --- 顯示圖例 ---

plt.legend(loc="lower right") # 圖例位置：右下角
```

這段程式碼的主要功能：

1. 多類別 ROC 曲線視覺化：

- 為每個類別（Normal、Ischemia、Hemorrhagic）繪製獨立的 ROC 曲線
- 使用不同顏色區分各類別
- 顯示每個類別的 AUC 值

2. 對角線基準線：

- 黑色虛線代表隨機猜測（AUC = 0.5）

- ROC 曲線越靠近左上角，性能越好
- 如果曲線接近或低於對角線，表示模型性能很差

3. ROC 曲線解讀：

- 左上角：高 TPR、低 FPR（理想狀態）
- 對角線上：TPR = FPR（隨機猜測）
- 曲線下面積（AUC）：
 - 1.0：完美分類
 - 0.5：隨機猜測
 - < 0.5：比隨機猜測還差

4. 醫療應用意義：

- 可以比較模型在不同中風類型上的診斷能力
- 幫助醫療人員了解模型在哪種類型上表現較好或較差

```
# --- 儲存 ROC-AUC 曲線圖 ---
```

```
# 組合完整的保存路徑
```

```
roc_save_path = os.path.join(plot_save_dir, "roc_auc_curve.png")
```

```
# 將 ROC-AUC 曲線圖保存為 PNG 檔案
```

```
plt.savefig(roc_save_path)
```

```
# 關閉圖表，釋放記憶體
```

```
plt.close()
```

```
# 顯示保存成功訊息
```

```
print(f"ROC-AUC curve plot saved to '{roc_save_path}')
```

```
# --- 準備將 AUC 分數寫入報告 ---

report_lines_to_add.append("\n--- ROC-AUC Scores ---") # 添加 AUC 分數區
塊標題


# 遍歷每個類別，將其 AUC 值加入報告

for i in range(n_classes):

    report_lines_to_add.append(f" {class_names[i]} AUC: {roc_auc[i]:0.4f}")

    # 格式：類別名稱 + AUC 值（保留 4 位小數）


# 添加 Micro-average AUC 到報告

report_lines_to_add.append(f" Micro-average AUC: {roc_auc['micro']:.4f}")


# === 3. 儲存包含所有指標的完整報告 ===

# 定義報告檔案的保存路徑

report_filepath = os.path.join(output_dir, "classification_report.txt")


# 以寫入模式開啟檔案

with open(report_filepath, 'w') as f:

    # with 語句確保檔案操作完成後自動關閉


    # --- 寫入基本評估指標 ---

    # 寫入測試集準確率（轉換為百分比，保留 2 位小數）
```

```
f.write(f"Test Set Accuracy: {val_accuracy*100:.2f}%\n")

# 寫入測試集損失值（保留 4 位小數）

f.write(f"Test Set Loss: {val_loss:.4f}\n\n")

# \n\n: 兩個換行，與下一個區塊之間留空行
...
```

這段程式碼的主要功能：

1. ****保存 ROC-AUC 圖表****：

- 將視覺化結果保存為圖片檔案
- 方便日後查看和用於報告、論文

2. ****整理 AUC 分數****：

- 收集每個類別的 AUC 值
- 包含 **micro-average AUC**（整體性能指標）
- 準備寫入文字報告

3. ****建立文字報告****：

- 創建結構化的評估報告檔案
- 先寫入基本指標（準確率和損失值）
- 為後續的詳細指標預留空間

4. ****報告檔案的用途****：

- 永久保存模型評估結果
- 便於比較不同模型或不同訓練階段的性能
- 可用於撰寫研究報告或專題文件

****報告檔案範例內容**：**

...

Test Set Accuracy: 95.32%

Test Set Loss: 0.1543

[後續會添加分類報告、特異度、NPV、AUC 等指標]

--- 寫入標準分類報告 ---

f.write("Classification Report:\n") # 寫入分類報告的標題

f.write(report) # 寫入完整的分類報告內容

report 包含：Precision（精確率）、Recall（召回率）、F1-score、Support（支持度）

--- 寫入新增的評估指標 ---

遍歷之前準備好的額外指標列表

for line in report_lines_to_add:

f.write(line + "\n") # 將每一行寫入檔案，並在結尾加上換行符號

包含的指標：

- Specificity（特異度）：每個類別

- NPV（陰性預測值）：每個類別

- ROC-AUC 分數：每個類別 + Micro-average


```
# 顯示報告保存成功的訊息
```

```
print(f"Classification report (with new metrics) saved to '{report_filepath}')
```

```
# 顯示所有任務完成的訊息
```

```
print("\n--- All tasks completed and results saved. ---")
```

```
...
```

這段程式碼的主要功能：

1. ****整合完整報告****：

- 將標準分類報告（`sklearn` 的 `classification_report`）寫入檔案
- 追加自定義的額外指標（`Specificity`、`NPV`、`AUC`）

2. ****報告結構****：

```
...
```

```
Test Set Accuracy: XX.XX%
```

```
Test Set Loss: X.XXXX
```

```
Classification Report:
```

```
[精確率、召回率、F1-score 等標準指標]
```

```
--- Additional Metrics (One-vs-Rest) ---
```

```
Class: Normal
```

Specificity: X.XXXX

NPV: X.XXXX

[其他類別...]

--- ROC-AUC Scores ---

Normal AUC: X.XXXX

Ischemia AUC: X.XXXX

Hemorrhagic AUC: X.XXXX

Micro-average AUC: X.XXXX

3. 完成訊息：

- 確認報告已成功保存
- 通知所有評估任務已完成

4. 實用價值：

- 提供全面的模型性能評估
- 方便與其他模型或研究進行比較
- 可直接用於專題報告或論文

這個完整的報告檔案包含了評估一個醫療影像分類模型所需的所有關鍵指標！